Python Training — Day 1

Input / Output | f-Strings | For Loop | List | Tuple

---

1. Taking Input from the User

Until now we have been using print() to show output. Now let's go one step further — we will also take input from the user. Real programs always have a conversation with the user: they ask something, the user types an answer, and the program responds.

1.1 The input() Function

When Python sees input(), it stops the program and waits for the user to type something and press Enter. Whatever the user types gets stored as a string.

Syntax:

```
variable name = input("Your message here: ")
```

Example 1 — Ask a name and greet the user:

```
name = input("Enter your name: ")
print("Hello", name)
```

Output (user types: ksrm)

```
Enter your name: ksrm
Hello ksrm
```

> Note: input() ALWAYS returns a string — even if the user types a number. So if the user types 25, Python stores it as "25" (text), not the number 25. You need to convert it using int() or float().

1.2 f-Strings — Mixing Variables into Text Neatly

An f-string is the cleanest way to include variables inside a sentence. Put f before the opening quote, then wrap any variable in curly braces { } right inside the string.

Syntax:

```
print(f"some text {variable} more text")
```

Example 2 — Greeting with f-string:

```
name = input("Enter your name: ")
print(f"Hello, {name}! How are you?")
```

Output:

```
Enter your name: ksrm
Hello, ksrm! How are you?
```

Example 3 — Adding two numbers (note: we convert input to int):

```
a = int(input("Enter first number: "))
b = int(input("Enter second number: "))
print(f"The sum of {a} and {b} is {a + b}")
```

Output:

```
Enter first number: 10
Enter second number: 5
The sum of 10 and 5 is 15
```

> Note: We use int() because input() gives us a string. Without int(), Python would join "10" + "5" as text and give "105" instead of 15.

Practice — Input & f-Strings
1. Ask the user to enter their city name. Print: Welcome to {city}!
2. Ask the user to enter two numbers. Print their product using an f-string.
3. Ask the user for their name and age. Print: Hi {name}, you will be {age+1} next year!
4. Ask the user to enter a temperature in Celsius. Convert it to Fahrenheit (F = C*9/5 + 32) and print the result using an f-string.

---

2. For Loop with range()

A for loop repeats a block of code a specific number of times. Think of it as giving Python a clear instruction: go from here to there, do this task each step.

2.1 The range() Function

range() generates a sequence of numbers for the loop to go through. It takes up to 3 arguments:

- start — where the sequence begins (included)
- stop — where the sequence ends (NOT included — Python stops just before this)
- step — how much to add each time (default is +1)

Syntax:

```
for variable in range(start, stop, step):
```

Example 1 — Print hi 3 times (range(3,6) produces 3, 4, 5 — three values):

```
for i in range(3, 6):
    print("hi")
```

Output:

```
hi
hi
hi
```

Example 2 — Print numbers 1 to 5:

```
for i in range(1, 6):
    print(i)
```

Output:

```
1
2
3
4
5
```

Example 3 — Count by 2s (even numbers from 0 to 10):

```
for i in range(0, 11, 2):
    print(i)
```

Output:

```
0
2
4
6
8
10
```

Example 4 — Countdown from 5 to 1 (step = -1):

```
for i in range(5, 0, -1):
    print(i)
```

Output:

```
5
4
3
2
1
```

> Note: The loop variable i holds the current number in each round. You can name it anything (i, j, num, count), but i is the common convention.

Practice — For Loop

1. Print your name 5 times using a for loop.
2. Print all numbers from 1 to 10.
3. Print the multiplication table of any number the user enters (1x to 10x).
4. Print all odd numbers between 1 and 20.
5. Print a countdown from 10 down to 1, then print Blast off!
6. Print the sum of all numbers from 1 to 100 using a for loop.

3. List

A List is a data structure — it lets you store multiple values under one variable name, in order. Think of it like a shopping list: one list, many items.

3.1 List Powers

Four key properties that define a List:

- **Mutable —** You CAN change, add, or remove items after the list is created.
- **Duplicates —** The same value can appear more than once. [1, 2, 2, 3] is perfectly valid.
- **Ordered —** Items stay in the order you put them. You access them using index numbers (positions).
- **Heterogeneous —** One list can hold strings, numbers, booleans — all mixed together.

3.2 Creating a List

Use square brackets [ ] and separate items with commas.

Examples:

```
fruits   = ["apple", "banana", "cherry"]
numbers  = [10, 20, 30, 40]
mixed    = ["Alice", 25, True, 3.14]   # heterogeneous
empty    = []                          # empty list
```

3.3 Indexing and Slicing

Works exactly the same as strings. Each item has a position number starting from 0.

Indexing:

```
fruits = ["apple", "banana", "cherry"]
print(fruits[0])    # apple    (first item)
print(fruits[1])    # banana   (second item)
print(fruits[-1])   # cherry   (last item)
```

Slicing:

```
numbers = [10, 20, 30, 40, 50]
print(numbers[1:4])   # [20, 30, 40]  — index 1 up to (not including) 4
print(numbers[:3])    # [10, 20, 30]  — from start to index 3
print(numbers[2:])    # [30, 40, 50]  — from index 2 to end
```

3.4 Modifying a List (Mutability in Action)

You can directly change any element using its index — something you cannot do with strings.

Example:

```
# Define a list
numbers = [10, 20, 30]

```

```
# Change the value at index 1
numbers[1] = 99

print(numbers)    # [10, 99, 30]
```

> Note: Try doing the same with a string: name = "hello"; name[0] = "H" — Python gives a TypeError. That shows the key difference: lists are mutable, strings are not.

3.5 Traversing a List

Method 1 — Loop directly through values:

```
fruits = ["apple", "banana", "cherry"]

for fruit in fruits:
    print(fruit)
```

Output:

```
apple
banana
cherry
```

Method 2 — Loop using index numbers:

```
for i in range(len(fruits)):
    print(i, fruits[i])
```

Output:

```
0 apple
1 banana
2 cherry
```

> Note: len() returns the total number of items in the list. len(fruits) here is 3.

3.6 List Methods

Methods are built-in tools that come with every list. You use them like this: list_name.method_name()

All methods at a glance:

```
numbers = [5, 2, 9, 1, 5, 6]    # starting list

numbers.append(10)              # Adds 10 to the END
numbers.insert(2, 15)           # Inserts 15 at index 2
numbers.extend([20, 25, 30])    # Adds multiple items to the END
numbers.remove(5)               # Removes the FIRST occurrence of 5
popped = numbers.pop(3)         # Removes & returns item at index 3
idx    = numbers.index(6)       # Returns the index of value 6
```

```
count  = numbers.count(5)       # Counts how many times 5 appears
numbers.sort()                  # Sorts ascending
numbers.reverse()               # Reverses the current order
copy   = numbers.copy()         # Makes an independent copy
numbers.clear()                 # Empties the list  →  []
```

append() — Add one item to the end:

```
cart = ["milk", "bread"]
cart.append("eggs")
print(cart)   # ['milk', 'bread', 'eggs']
```

insert() — Add an item at a specific position:

```
cart = ["milk", "bread", "eggs"]
cart.insert(1, "butter")
print(cart)   # ['milk', 'butter', 'bread', 'eggs']
```

remove() — Remove the first matching item:

```
nums = [1, 2, 3, 2, 4]
nums.remove(2)
print(nums)   # [1, 3, 2, 4]
```

pop() — Remove by index and return the value:

```
nums = [10, 20, 30, 40]
removed = nums.pop(2)   # removes index 2 (value 30)
print(removed)   # 30
print(nums)      # [10, 20, 40]
```

sort() and reverse():

```
nums = [3, 1, 4, 1, 5, 9]
nums.sort()             # ascending
print(nums)    # [1, 1, 3, 4, 5, 9]

nums.reverse()
print(nums)    # [9, 5, 4, 3, 1, 1]
```

Practice — Lists

1. Create a list of 5 favourite movies and print the full list.
2. Print only the first and last movie using indexing.
3. Add one more movie at the end using append(). Print the updated list.
4. Remove the second movie using remove(). Print the updated list.
5. Sort the list alphabetically and print it.
6. Traverse the list with a for loop and print each movie on a new line.
7. Create a list of 5 numbers. Print the largest number using sort().

4. Tuple

A Tuple is like a List — it stores multiple values in order. The ONE major difference: once a tuple is created, you CANNOT change it. It is locked forever.

Think of a tuple like your date of birth — set once, never changes.

4.1 Tuple Powers

- **Immutable —** Values CANNOT be changed after creation.
- **Duplicates —** Same values are allowed.
- **Ordered —** Items keep their insertion order. Access using index.
- **Heterogeneous —** Can hold different data types together.

4.2 Creating a Tuple

Use regular brackets ( ) with items separated by commas.

Examples:

```
coords   = (10.5, 20.3)                # GPS coordinates — perfect for tuple
colors   = ("red", "green", "blue")
person   = ("Alice", 25, "Chennai")    # mixed types
repeated = (1, 2, 2, 3, 3, 3)          # duplicates allowed
```

Indexing and slicing — same as lists:

```
colors = ("red", "green", "blue")
print(colors[0])    # red
print(colors[-1])   # blue
print(colors[1:3])  # ('green', 'blue')
```

4.3 What Happens When You Try to Change a Tuple

```
colors = ("red", "green", "blue")
colors[0] = "yellow"   # This gives an ERROR
# TypeError: 'tuple' object does not support item assignment
```

> Note: This is the intended behaviour. Use a tuple when you want data to stay fixed — like days of the week, month names, or coordinate pairs.

4.4 Traversing a Tuple

```
colors = ("red", "green", "blue")

for color in colors:
    print(color)
```

Output:

```
red
green
blue
```

4.5 Tuple Methods (Only 2)

Because tuples are immutable, they have only 2 methods:

index() — find where a value is:

```
t = (5, 2, 9, 1, 5, 6)
print(t.index(9))     # 2
```

count() — count how many times a value appears:

```
t = (5, 2, 9, 1, 5, 6)
print(t.count(5))     # 2
```

4.6 List vs Tuple — At a Glance

Feature	List [ ]	Tuple ( )
Mutable (changeable)	Yes	No
Duplicates allowed	Yes	Yes
Ordered	Yes	Yes
Heterogeneous	Yes	Yes
Methods	11 methods	2 methods (index, count)
Best used when	Data will change	Data stays fixed

Practice — Tuples

1. Create a tuple of 4 Indian state names and print it.
2. Print the second and last state using indexing.
3. Loop through the tuple and print each state on a new line.
4. Count how many times 'Goa' appears in: ('Kerala', 'Goa', 'Goa', 'Tamil Nadu')
5. Find the index of 'Tamil Nadu' in the tuple above.
6. Try changing a value in a tuple. Write down the error message you get.

Day 1 — Mixed Practice Problems

Try these on your own. Use everything you learned today.

1. Ask the user to enter 5 numbers one by one. Store them in a list. Print the list, the sorted version, and the total sum.
2. Create a tuple of weekday names. Print each day using a for loop. Also print the day at index 3.
3. Ask the user to enter a name and a number. Print the name that many times using a for loop.
4. Create a list of 5 numbers. Find and remove the largest number, then print the updated list.
5. Create a mixed list with your name, age, and city. Print each item using a for loop with f-string